

SpecDrift-Bench: A Fault-Injection Benchmark for Evaluating LLM-Based Specification-Drift Detection

K Nandesh
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
127018025@sastra.ac.in

S Soundar
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
127018056@sastra.ac.in

N Sudharsan
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
127018059@sastra.ac.in

Vivekanandan S J
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
vivekanandansj@sastra.ac.in

Abstract—Large language models now write a substantial share of production code, and they introduce a failure mode that functional testing does not catch: specification drift, in which an implementation quietly stops satisfying a requirement it once met while the project's own test suite continues to pass. Tools that claim to detect drift are evaluated by the issues they report rather than by the issues they miss or invent, so their trustworthiness is unknown. This paper presents SpecDrift-Bench, a fault-injection benchmark that treats drift detection as a classification problem in which false alarms are weighted as heavily as misses. We define nine drift categories, including comment decoys and reverse-direction unauthorised scope creep, together with three categories of negative control, and construct 86 validated cases by injecting faults into three working Python projects governed by 40 numbered rules. Every case records whether the host project's own tests still pass: 22 percent of injected drift escapes testing, and unauthorised scope creep escapes it in every case. Evaluating a two-stage detector with independently instrumented retrieval and verification stages, we find that retrieval raises recall from 0.17 to 0.58 and the false-alarm rate from 0.04 to 0.63, and that the false alarms surviving an evidence-citation requirement are confidently wrong rather than unsupported.

Keywords—specification drift, benchmark construction, fault injection, LLM-based verification, false-positive analysis, empirical software engineering

I. INTRODUCTION

Large language models have moved from autocompletion to authorship. Code is generated, refactored and extended by assistants that read a natural-language requirement once and then operate on the repository for weeks afterwards. The requirement does not travel with the code. What results is specification drift: the implementation diverges from the requirement that justified it, and nothing in the ordinary development loop announces the divergence.

Drift is not the same as a bug. A bug breaks behaviour the tests describe, and the tests go red. Drift breaks behaviour the specification describes but the tests happen not to cover, so the suite stays green. In the benchmark presented here, 22 percent of all injected drift leaves the host project's own test suite passing, and behaviour added beyond what the specification authorises survives the tests in every single case: those tests were written before the behaviour existed, so they cannot fail on it.

Three families of tooling address the problem and none measures its own reliability. Static analysers and keyword-based checks match vocabulary rather than behaviour; a rule forbidding a fourth concurrent loan is neither satisfied nor violated by the presence of the word "loan". Spec-driven toolkits such as GitHub Spec Kit [2] and AWS Kiro [3]

validate adherence at generation time and never again, leaving every subsequent edit unchecked. LLM-based checkers such as Semcheck [4] do verify continuously, but their published evaluations report issue counts, which have no denominator: an issue count cannot distinguish a checker that finds nine violations out of ten from one that finds nine out of ninety, and says nothing about how often the checker reports a violation in code that is correct. Without that second number a drift detector cannot be deployed, because a checker that flags two thirds of untouched code is noise however much real drift it also catches.

There is good reason to expect that second number to be bad. Jin and Chen [1] show that LLMs asked to judge whether correct code conforms to its stated requirement systematically overcorrect, and that the rate rises when the prompt asks the model to explain itself and propose a correction. Their study establishes the failure on function-level generation benchmarks. This paper asks what happens at repository scale, against a numbered specification document, on code the model must first locate for itself. We contribute a twelve-category drift taxonomy including reverse-direction scope creep, a benchmark of 86 validated and test-escape-labelled cases, a two-stage detector whose misses are attributed to either retrieval or reasoning, a scoring protocol in which a flag counts only when it names the targeted rule, and the empirical finding that retrieval triples recall while multiplying false alarms fifteenfold.

II. RELATED WORK

A. Conformance Checking and Its Evaluation

Requirements engineering distinguishes verification, which asks whether the system was built right, from validation, which asks whether the right system was built [8]. Automated testing addresses the first against an executable encoding of the requirement. Specification drift lives in the residue: requirements stated in prose, implemented once and never encoded as a test. Spec Kit [2] and Kiro [3] constrain the first commit and nothing after it; Semcheck [4] re-checks continuously but reports issue counts on real repositories, where ground truth is unavailable by construction. SpecDrift-Bench supplies the missing denominators by injecting known faults into code whose correct behaviour is known. HumanEval [5], MBPP [6] and EvalPlus [7] measure whether generated code is functionally correct, and SWE-bench [9] whether an agent resolves a real issue; none measures surveillance, that is, whether a tool notices after the fact that existing code has stopped matching an existing written requirement.

B. Reliability of LLM Conformance Judgement

The base study for this work is Jin and Chen [1], who evaluate five representative LLMs on requirement conformance judgement over HumanEval and MBPP. GPT-4o reaches a false negative rate of 26.2 percent on HumanEval and 35.9 percent on MBPP under a plain verdict prompt, rejecting correct implementations at those rates. The striking result is what happens when the prompt is enriched: requiring the model to explain its verdict and propose a repair raises the HumanEval false negative rate to 73.2 percent. Prompt elaboration, normally a reliability intervention, makes the judge markedly worse. The authors trace most false rejections to unsupported "logic error" assertions and propose a Fix-guided Verification Filter that treats the model's proposed fix as executable counterfactual evidence.

Three design decisions in our detector follow directly. The verification prompt never asks for a fix. A drift verdict is accepted only if it quotes the clause it claims is violated and supplies a concrete counterexample. And a possible risk is stated, in the prompt itself, not to be a violation. Our contribution is complementary rather than competing: they characterise the failure on function-level benchmarks and mitigate it with execution, whereas we measure whether it persists at repository scale, per drift category, once the model must also locate the code it is judging.

III. BENCHMARK DESIGN

A drift benchmark needs code whose requirements are written down, atomic and testable. We therefore authored three complete Python services rather than mining repositories, because mined code has no trustworthy rule-level ground truth: library, a loan and catalogue service with 14 rules and 28 cases; wallet, a transfer and fee service with 13 rules and 29 cases; and ratelimiter, a token-bucket throttle with 13 rules and 29 cases. Each ships a specification of numbered rules, a source tree and one test per rule, and each passes its own suite before any injection. A representative rule reads: R06, a member **MUST NOT** hold more than 3 active loans at once. Rules are deliberately atomic, so a verdict on one is not entangled with a verdict on another, and deliberately quantified, so a violation has a witness.

Table I defines nine drift categories and three control categories. Eight drift categories are forward: existing behaviour is changed so that it no longer satisfies a rule. One, D8, is reverse: behaviour is added that no rule authorises, such as an undocumented loyalty discount inside a transfer. Reverse drift is the category existing tooling is structurally unable to see, because there is no rule to check the new code against. The negative controls carry as much design weight as the drift cases. N2, semantics-preserving refactoring, is the largest single category at 18 cases, because it is the condition under which a detector is most likely to hallucinate a violation: the code has visibly changed and its behaviour has not.

TABLE I. DRIFT AND CONTROL TAXONOMY

ID	Category	Direction	n
D1	Boundary shift	forward	8
D2	Value / constant drift	forward	9
D3	Omitted check	forward	6
D4	Weakened condition	forward	6
D5	Sequence violation	forward	6
D6	Error-handling drift	forward	6
D7	Comment decoy	forward	6
D8	Unauthorised scope creep	reverse	6
D9	Output contract drift	forward	6
N1	Unchanged code	control	3
N2	Semantics-preserving refactor	control	18
N3	Benign decoy	control	6

Each case is a declarative YAML entry naming the host project, the category, the ground-truth label, the targeted rules, the file, a find and replace pattern, and the qualified name of the gold code symbol a detector would have to retrieve. The builder copies the host project into an isolated workspace and applies the substitution, which must match exactly once; zero or two matches fail the build rather than producing a silently wrong case. The validator then compiles the patched file and runs the host project's own suite inside the workspace. A negative control whose tests fail is marked invalid and excluded, on the reasoning that a refactor which changes behaviour is not a refactor and would corrupt the false-alarm rate. All 86 cases validate.

Every drift case additionally records whether the host suite still passes, which turns the paper's motivating claim into a measurement. Table II gives the result. Two readings matter. Testing is a partial defence, not an absent one: 78 percent of injected drift is caught by tests never written with drift in mind. But the residue is not randomly distributed. Unauthorised scope creep escapes in all six cases and output contract drift in half, because both concern behaviour the suite does not assert about. These are exactly the categories for which a specification-level detector is the only available defence.

TABLE II. DRIFT THAT SURVIVES THE HOST PROJECT'S OWN TESTS

Category	n	Escapes	Rate
D1 Boundary shift	8	1	0.12
D2 Value / constant drift	9	1	0.11
D3 Omitted check	6	0	0.00
D4 Weakened condition	6	0	0.00
D5 Sequence violation	6	2	0.33
D6 Error-handling drift	6	0	0.00
D7 Comment decoy	6	0	0.00
D8 Unauthorised scope creep	6	6	1.00
D9 Output contract drift	6	3	0.50
All drift	59	13	0.22

IV. DETECTORS AND PROTOCOL

SpecGuard, the detector the benchmark was built to examine, is deliberately separated into two stages so that failures can be attributed. Stage 1, rule retrieval, parses the specification into numbered rules and the source tree into chunks at function, method, class and module granularity using the abstract syntax tree, so a chunk is always syntactically complete. Rules and chunks are embedded into a shared space and ranked by cosine similarity, and the top k chunks per rule are forwarded, with k set to 3 throughout. One refinement proved important: when a retrieved chunk references a module-level constant, the chunk defining that constant is appended. A rule stating that the loan period must be 14 days, judged against the expression `day plus LOAN_PERIOD_DAYS` with the constant out of view,

produces a verifier that is not so much wrong as under-informed; adding constant resolution cut the false-alarm rate on the offline stand-in provider from 0.67 to 0.33.

Stage 2, semantic verification, asks the model about exactly one rule at a time, showing only the retrieved excerpts with their true line numbers, and requires a JSON verdict of COMPLIANT, DRIFT or UNCERTAIN with a confidence, a quoted violated clause, an evidence span and a counterexample. Following [1], the prompt forbids fix suggestions and states that comments are claims rather than behaviour. An evidence gate is then applied mechanically outside the model: a DRIFT verdict lacking either a quoted clause or a concrete counterexample is downgraded to UNCERTAIN before scoring.

Two baselines isolate what each stage contributes. The keyword baseline tokenises each rule and searches all source text including comments, establishing the floor any semantic method must clear; its inclusion of comments is what makes a comment decoy fool it, which is the point of having it. The whole-file baseline places the entire codebase and specification into a single prompt and asks for a verdict on every rule at once, which isolates the contribution of retrieval because it differs from SpecGuard in that respect alone. Responses are cached on a hash of the question, so a full 86-case SpecGuard run costs 414 model calls rather than the 1,146 rule-case pairs it covers; temperature is fixed at zero and retrieval ties break by chunk identifier.

The positive class is drift, and scoring is stricter than conventional binary classification in one respect that matters for trustworthiness. A drift case is a true positive only when the detector flags one of the rules the injection actually targeted; a flag landing on an unrelated rule earns nothing and is tallied separately as spurious, because a detector that is right by accident offers a reviewer nothing to act on. D8 cases name no target rule, since no rule authorises the added behaviour, so any flag counts. Any flag on a negative control is a false alarm, reported alongside precision rather than folded into it. UNCERTAIN counts as not flagged, so the evidence gate can only cost recall. Each miss is attributed: if the injected chunk was never retrieved the miss belongs to retrieval, and if it was retrieved and the verdict was still COMPLIANT the miss belongs to reasoning.

V. RESULTS

Results come from a single run of each detector over all 86 cases with gpt-4o-mini at temperature zero, supplemented by retrieval and keyword measurements reproduced for this paper. They are preliminary at n equal to 1, and are reported because their shape, not their precision, is the finding.

Stage 1 in isolation, across 64 rule queries with a gold symbol, achieves Recall@1 of 0.27, Recall@3 of 0.78 and Recall@5 of 0.92. At the operating point used throughout, roughly one injected chunk in five is never shown to the verifier, which places a hard ceiling on achievable recall. Per category at k equal to 3, retrieval is perfect on weakened conditions, sequence violations and output contract drift, and weakest on unauthorised scope creep at 0.33. That weakness is structural rather than incidental: retrieval ranks code by similarity to a rule, and code that no rule authorises is by construction similar to no rule. Omitted checks, at 0.50, are second weakest for the related reason that deleted code leaves nothing behind to retrieve.

TABLE III. END-TO-END DETECTOR COMPARISON ($N = 1$, TEMPERATURE 0)

Detector	P	R	F1	FAR	Unc.	Calls
SpecGuard	0.67	0.58	0.62	0.63	0.06	252
Whole-file	0.91	0.17	0.29	0.04	0.01	86
Keyword	0.33	0.08	0.14	0.37	0.00	0

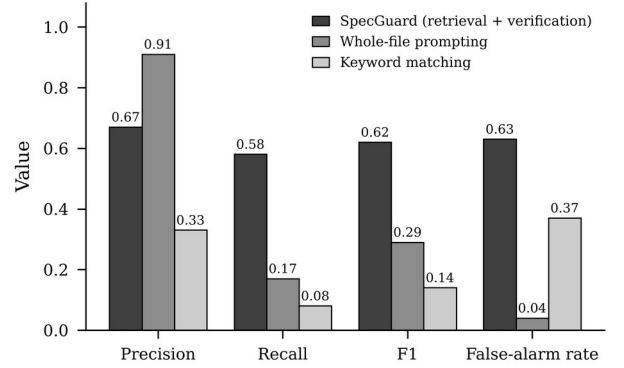


Fig. 1. End-to-end detector comparison. The two LLM-based detectors fail in opposite directions: retrieval buys recall and pays for it in false alarms.

Table III and Fig. 1 give the end-to-end comparison, and the contrast between the two LLM-based detectors is the clearest result in the paper, because they differ only in whether a retrieval stage is present. Retrieval more than triples recall, from 0.17 to 0.58: showing the model a short, relevant excerpt is what lets it see a violation at all. The same intervention raises the false-alarm rate from 0.04 to 0.63, so SpecGuard flags nearly two thirds of the untouched and semantics-preserving controls. Whole-file prompting is quiet, precise and largely blind, missing five of the nine drift categories outright. The keyword baseline establishes the floor at recall 0.08 and false-alarm rate 0.37, both deaf and noisy, with 18 spurious flags and zero recall on weakened conditions, sequence violations, error-handling drift, comment decoys and output contract drift.

SpecGuard's per-category profile has a consistent shape. It is strongest where a rule states something directly checkable against a value in the code: value and constant drift and weakened conditions are both caught in every case. It is weakest on sequence violations and error-handling drift, at 0.33 each, which require reasoning about the order in which operations occur and about which exception propagates, rather than comparing a stated literal against a written one. The gradient runs from comparison to reasoning, and it is steep.

The evidence gate works as designed and does not solve the problem. Requiring a quoted clause and a concrete counterexample removes unsupported drift claims; it does not remove wrong ones. In one false alarm on unmodified code, against a rule stating that a new bucket MUST start full, the model reported that the bucket yields tokens equal to 0.0, misreading a dataclass field declared on screen as tokens: float = BURST_LIMIT. In another, against a rule stating that an allowed request MUST consume exactly 1 token, the model's counterexample was a transition from 1.5 tokens to 0.5 tokens, which is consuming exactly one token and therefore confirms the rule it was offered as a violation of. Both arrived with a quoted clause, a well-formed counterexample and a confidence of 0.9, and both passed the gate.

VI. DISCUSSION AND THREATS TO VALIDITY

The headline trade-off is counter-intuitive for anyone expecting retrieval-augmented generation to improve a judge

uniformly. A model given the whole codebase answers conservatively and is nearly always right when it does answer, because it mostly does not answer. A model given a short excerpt answers readily, catches three times as much real drift, and invents violations in correct code. Retrieval does not make the model a better reader; it makes it a more willing one, and any deployment must budget for that willingness.

Requiring evidence eliminates the unsupported assertion that Jin and Chen [1] identify as dominant in their setting, and leaves behind a second class: the supported but false assertion, in which every required artefact is present and the content of one is wrong. Syntactic evidence requirements cannot catch this, because the defect is semantic. Only execution can, which is why the two lines of work converge: their Fix-guided Verification Filter executes the model's proposed repair, and our planned adversarial pass must execute the model's proposed counterexample. We cannot adopt their mechanism directly, since asking for a fix is exactly the prompt elaboration their results show to be harmful, so the counterexample rather than the fix must carry the executable burden. At a false-alarm rate of 0.63 no team would leave this detector enabled in continuous integration, and reporting that number is the point of the exercise; the practical path is not a better prompt but an abstention policy routing low-confidence verdicts to human review. The two examples above warn that calibration is not guaranteed, since both wrong verdicts carried confidence 0.9.

Several limits bound these conclusions. Cases are synthetic injections rather than drift observed in the wild, which buys rule-level ground truth at the cost of realism. Three host projects, 40 rules and one language limit generalisation. Detector results come from a single run of one model at temperature zero, so no variance is reported; the differences between detectors are large enough that their direction is unlikely to reverse, but their magnitudes are provisional. Retrieval results use a deterministic local hashing embedder rather than a trained sentence encoder [10], so Stage 1 numbers are a lower bound. Finally, requiring a flag to name the targeted rule is stricter than case-level detection and depresses recall relative to evaluations that count any flag on a drifted file; we regard the stricter measure as the one matching how a reviewer would use the output.

VII. CONCLUSION AND FUTURE WORK

SpecDrift-Bench measures what the drift-detection literature has reported around: not how many issues a checker raises, but how many it misses, how many it invents, and in which categories. On 86 validated cases the answer for a retrieval-augmented LLM detector is a recall of 0.58 bought at a false-alarm rate of 0.63, with the remaining false alarms confidently argued rather than unsupported. Twenty-two percent of the injected drift passes the host projects' own tests, and unauthorised scope creep passes them always, so the

problem the detector addresses is real even though the detector is not yet trustworthy.

Four extensions are scheduled, each with a slot already present in the framework. An adversarial second pass will execute the counterexample attached to every drift verdict and retain the verdict only if the code behaves as alleged. A reverse-direction authorisation pass will iterate over code looking for an authorising rule, rather than over rules looking for code, which is the only way to lift D8 retrieval above 0.33. Repeated runs across model scales, with Semcheck added as a third baseline, will replace single-run point estimates with variance. And confidence calibration will be measured to derive the abstention threshold deployment requires. The benchmark will be expanded to ten host projects and approximately 250 cases over the same taxonomy. The framework, case manifests, host projects and evaluation harness are available at [11].

ACKNOWLEDGMENT

The authors thank Dr. Vivekanandan S J for his guidance throughout this work, and the School of Computing, SASTRA Deemed to be University, for the computing resources used in the experiments.

REFERENCES

- [1] H. Jin and H. Chen, "Are LLMs reliable code reviewers? Systematic overcorrection in requirement conformance judgement," *Automated Software Engineering*, 2026, doi: 10.1007/s10515-026-00638-5.
- [2] GitHub, "Spec Kit: toolkit for spec-driven development," 2025. [Online]. Available: <https://github.com/github/spec-kit>
- [3] Amazon Web Services, "Kiro: the AI IDE for spec-driven development," 2025. [Online]. Available: <https://kiro.dev>
- [4] ReJot Labs, "Semcheck: semantic checking of implementation against specification using LLMs," 2025. [Online]. Available: <https://github.com/rejot-dev/semcheck>
- [5] M. Chen et al., "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.
- [6] J. Austin et al., "Program synthesis with large language models," *arXiv:2108.07732*, 2021.
- [7] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, International Organization for Standardization, 2018.
- [9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: can language models resolve real-world GitHub issues?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- [10] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [11] K. Nandesh, S. Soundar, and N. Sudharsan, "SpecDrift-Bench," 2026. [Online]. Available: <https://github.com/nandeshkanagaraju/specdrift-bench>